ТОО ”Колледж Хекслет”

Лабораторная работа №1

Аутентификация пользователей

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_4. Аутентификация пользователей.

Цель: Создать модуль аутентификации Internet Shop с регистрацией и входом пользователя, безопасным хранением passwordHash и разделением Route, Controller и Service.

Условие: RO4_4/lesson1/ЛР-1.pdf

Требования задания

• В User существует поле passwordHash.

• Исходный password не сохраняется в PostgreSQL.

• Регистрация работает через POST /api/auth/register.

• Login работает через POST /api/auth/login.

• passwordHash отсутствует в ответах API.

• Неверный password возвращает 401.

• Неизвестный email возвращает такое же внешнее сообщение login.

• Auth Service не использует req и res.

• Auth Controller не содержит Prisma-запросов.

• Frontend может отправить login через fetch().

Ход выполнения

User дополнен passwordHash и role. Регистрация проверяет данные и уникальный email, хеширует пароль через Argon2 и возвращает безопасный select. Login проверяет verify(), не сравнивает новые хеши. Неверный пароль и неизвестный email дают одинаковую ошибку 401. HTML-форма отправляет fetch и показывает имя. Старые записи сохраняются миграцией и требуют назначения пароля; открытый пароль не создаётся.

Проект: RO4_4/lesson1

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

prisma/migrations/20261104000400_add_user_auth_fields/migration.sql

```sql
-- Существующие пользователи сохраняются. Их старые пароли неизвестны:
-- временный маркер не является паролем или допустимым Argon2-хешем.
ALTER TABLE "users" ADD COLUMN "password_hash" TEXT;
ALTER TABLE "users" ADD COLUMN "role" TEXT NOT NULL DEFAULT 'user';
UPDATE "users" SET "password_hash" = '!disabled-until-password-reset!' WHERE "password_hash" IS NULL;
ALTER TABLE "users" ALTER COLUMN "password_hash" SET NOT NULL;
```

Листинг 1. prisma/migrations/20261104000400_add_user_auth_fields/migration.sql

services/passwordService.js

```js
import * as argon2 from 'argon2';
export function hashPassword(password) { return argon2.hash(password); }
export function verifyPassword(hash, password) { return argon2.verify(hash, password); }
```

Листинг 2. services/passwordService.js

services/authService.js

```js
import { prisma } from '../config/prisma.js';
import { AppError } from '../errors/AppError.js';
import { hashPassword, verifyPassword } from './passwordService.js';
const safeUser = { id: true, name: true, email: true, role: true };
export function findUserById(id) { return prisma.user.findUnique({ where: { id }, select: safeUser }); }
export async function registerUser({ name, email, password }) {
    email = email.trim().toLowerCase();
    if (await prisma.user.findUnique({ where: { email } })) throw new AppError('Email already registered', 409);
    const passwordHash = await hashPassword(password);
    try { return await prisma.user.create({ data: { name: name.trim(), email, passwordHash }, select: safeUser }); }
    catch (error) { if (error.code === 'P2002') throw new AppError('Email already registered', 409); throw error; }
}
export async function authenticateUser(email, password) {
    const user = await prisma.user.findUnique({ where: { email: email.trim().toLowerCase() } });
    if (!user || !user.passwordHash.startsWith('$argon2') || !await verifyPassword(user.passwordHash, password)) return null;
    return { id: user.id, name: user.name, email: user.email, role: user.role };
}
```

Листинг 3. services/authService.js

controllers/authController.js

```
controllers/authController.js
```

```
import { registerUser, authenticateUser, findUserById } from
'../services/authService.js';

export async function register(req, res, next) {
    try { res.status(201).json({ user: await registerUser(req.body) }); }
    catch (error) { next(error); }
}
export async function login(req, res, next) {
    try {
        const user = await authenticateUser(req.body.email, req.body.password);
        if (!user) return res.status(401).json({ error: 'Invalid email or password' });
        res.json({ user  });
    } catch (error) { next(error); }
}
export async function getCurrentUser(req, res, next) {
    try {
        const user = await findUserById(req.user.id);
        if (!user) return res.status(401).json({ error: 'User not found' });
        res.json({ user });
    } catch (error) { next(error); }
}
```

Листинг 4. controllers/authController.js

public/js/login.js

```
public/js/login.js
```

```
const form = document.querySelector('#login-form');
const message = document.querySelector('#auth-message');
form.addEventListener('submit', async event => {
    event.preventDefault();
    try {
        const response = await fetch('/api/auth/login', { method: 'POST', headers: {
'Content-Type': 'application/json' }, body: JSON.stringify({ email:
form.elements.email.value, password: form.elements.password.value }) });
        const data = await response.json();
        if (!response.ok) throw new Error(data.error || data.message);
        message.textContent = `Здравствуйте, ${data.user.name}!`;
        form.elements.password.value = '';
    } catch (error) { message.textContent = `Ошибка: ${error.message}`; }
});
```

Листинг 5. public/js/login.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Миграции description/slug применены; повторный seed без дублей

• Атомарный заказ COMMIT/ROLLBACK; цена из БД; валидация позиций

• Регистрация/login; одинаковая ошибка для неверных credentials

• Argon2id, разные salt, пароль и хеш отсутствуют в ответах

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

POST /api/products → 201

DELETE /api/products/8 → 200

POST /api/orders → 201

POST /api/orders → 409

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 404

POST /api/auth/register → 201

POST /api/auth/register → 409

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/login → 200

POST /api/auth/login → 401

POST /api/auth/login → 401

GET /api/unknown → 404

Примеры ответов

POST /api/auth/register → 201

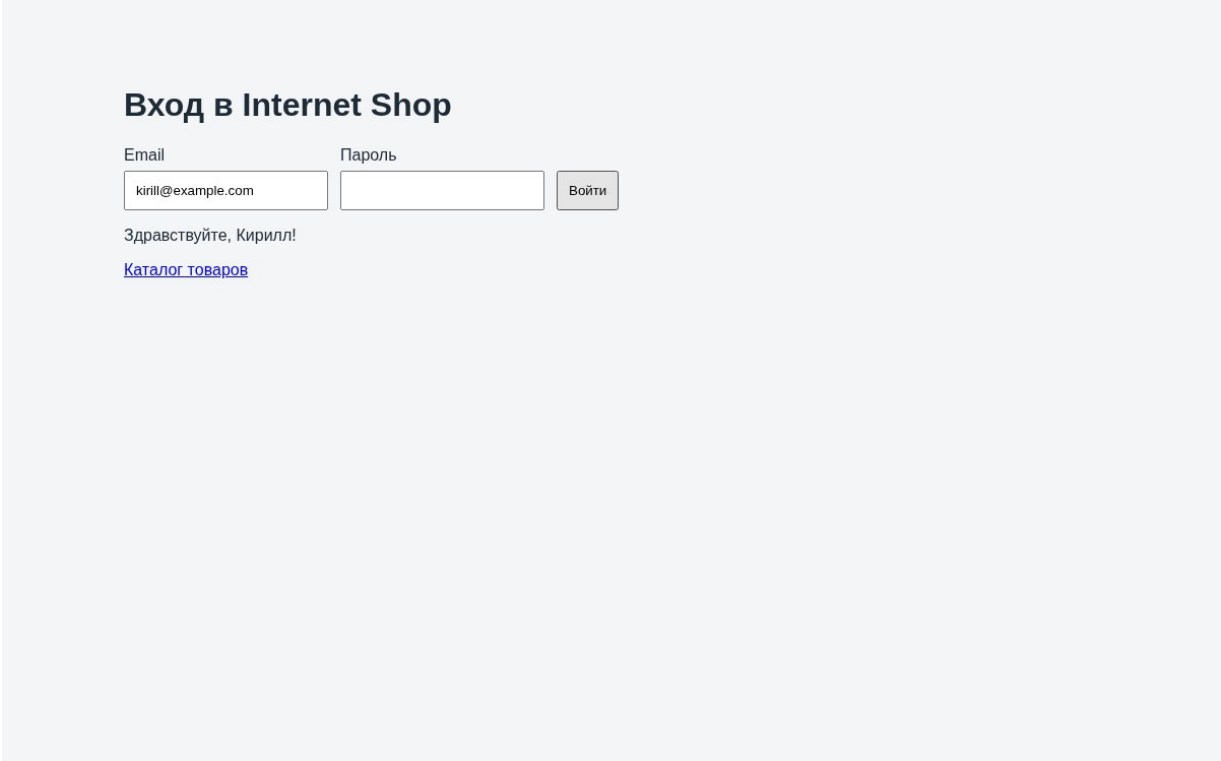

```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  }
}
```


POST /api/auth/login → 200


```
{
  "user": {
```

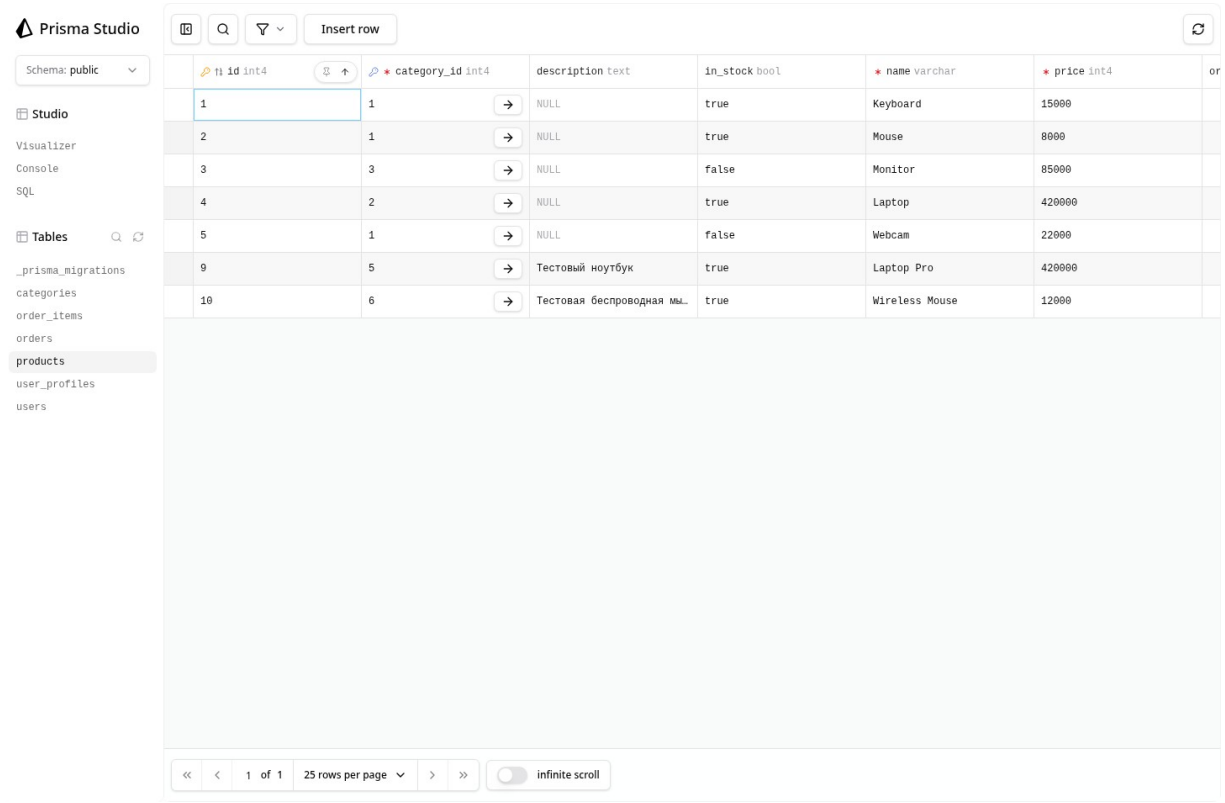
```
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  }
}
```

Проверка в браузере Chromium



Скриншот работающего приложения после выполнения действий через интерфейс.

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.